

KevinGym - Design Patterns

State Pattern & Builder Pattern

MERN Stack Implementation | QUT Assignment

Pattern 1: State Pattern

Feature: Membership Management

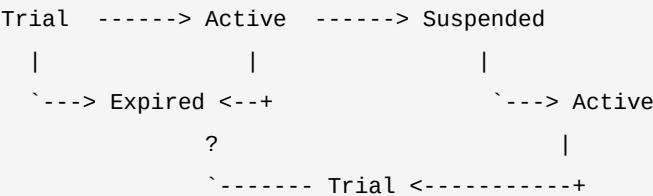
What is the State Pattern?

The State Pattern allows an object to change its behaviour when its internal state changes. The object appears to change its class. Instead of using large if/else chains, each state is encapsulated in its own class that defines what the object can and cannot do in that state.

How it is used in KevinGym

Every user has a membership status. Depending on the status, the member can or cannot book classes and access content. The status follows a strict transition graph - invalid jumps are rejected by the MembershipContext.

State Transition Diagram



Files Created

File	Role
backend/membership/MembershipState.js	Abstract base class - defines the interface
backend/membership/TrialState.js	canBook=true, canAccess=true, limited bookings
backend/membership/ActiveState.js	canBook=true, canAccess=true, full access
backend/membership/SuspendedState.js	canBook=false, canAccess=false
backend/membership/ExpiredState.js	canBook=false, canAccess=false, prompts renewal
backend/membership/MembershipContext.js	Holds current state, validates transitions
backend/controllers/membershipController.js	GET /api/membership/status, PUT /:id/transition
backend/routes/membershipRoutes.js	Route definitions

Code Example - MembershipContext

```
transitionTo(newStatusName) {
  const allowed = this.state.allowedTransitions();
  if (!allowed.includes(newStatusName)) {
    throw new Error(`Cannot transition from ${this.state.getName()} to ${newStatusName}`);
  }
  this.state = new STATE_MAP[newStatusName]();
}
```

Code Example - TrialState

```
class TrialState extends MembershipState {
  getName()          { return "trial"; }
  canBookClass()     { return true; }
  canAccessContent() { return true; }
  allowedTransitions() { return ["active", "expired"]; }
}
```

Observer Integration

Every membership transition emits a "membershipTransitioned" event on gymEvents. LoggerObserver logs it to the console and NotificationObserver stores a notification in MongoDB.

```
gymEvents.emit("membershipTransitioned", {
  name, email, from: previousStatus, to: ctx.getName()
});
```

Frontend Changes

- Member Panel - live colour-coded badge (blue=trial, green=active, yellow=suspended, red=expired)
- Member Panel - "Browse New Class" button disabled when canBookClass is false
- Admin Dashboard - Membership column in users table with coloured badges
- Admin Dashboard - Edit form dropdown shows only valid next states for selected user

Pattern 2: Builder Pattern

Feature: Workout Plan Builder

What is the Builder Pattern?

The Builder Pattern separates the construction of a complex object from its representation. A director (or client) calls a sequence of builder methods to configure the object step by step, then calls build() to get the final validated result. This prevents complex constructors with many parameters and makes the construction logic reusable and readable.

How it is used in KevinGym

Vendors assign personalised workout plans to members. A WorkoutPlanBuilder chains method calls to set the title, difficulty, duration, and exercises. Each step validates its input immediately. build() performs a final check before returning the completed plan object for saving to MongoDB.

Builder Chain (fluent interface)

```
const planData = new WorkoutPlanBuilder()
  .setTitle("8-Week Strength")
  .setDifficulty("intermediate")
  .setDuration(8)
  .addExercise({ name: "Squat",      sets: 4, reps: "8" })
  .addExercise({ name: "Bench Press", sets: 3, reps: "10" })
  .addExercise({ name: "Deadlift",   sets: 3, reps: "6" })
  .setNotes("Rest 90 seconds between sets.")
  .build();
```

Files Created

File	Role
backend/builder/WorkoutPlanBuilder.js	The builder class with fluent API
backend/models/WorkoutPlan.js	Mongoose schema (memberId, exercises[], etc.)
backend/controllers/workoutPlanController.js	Uses builder to construct plan before saving
backend/routes/workoutPlanRoutes.js	POST /api/workout-plans, GET /my, GET /member/:id
frontend/src/pages/WorkoutPlanPage.jsx	Member view - shows plan, difficulty badge, table

Code Example - WorkoutPlanBuilder

```
class WorkoutPlanBuilder {
  setTitle(title)      { this._title = title; return this; }
  setDifficulty(level) { /* validates enum */ this._difficulty = level; return this; }
  setDuration(weeks)   { /* validates integer */ this._durationWeeks = weeks; return this; }
  addExercise(exercise) { /* validates name */ this._exercises.push(exercise); return this; }
  setNotes(notes)      { this._notes = notes; return this; }

  build() {
    if (!this._title) throw new Error("Plan must have a title");
    if (this._exercises.length === 0) throw new Error("Add at least one exercise");
    return { title, difficulty, durationWeeks, exercises, notes };
  }
}
```

API Endpoints

Method	Endpoint	Who	Description
POST	/api/workout-plans	Vendor / Admin	Build and assign a plan to a member by email
GET	/api/workout-plans/my	Member	Fetch own most recent workout plan
GET	/api/workout-plans/member/:id	Vendor / Admin	Fetch any member's plan by user ID

Observer Integration

On successful plan creation, a "planCreated" event is emitted. LoggerObserver logs it and NotificationObserver stores a notification visible to all users.

```
gymEvents.emit("planCreated", { title: plan.title, memberName: member.name });
```

Frontend Changes

- Vendor Panel - Workout Plan Builder section at the bottom with dynamic exercise rows
- Vendor Panel - "Assign Plan" button sends to API, shows success/error message
- Member Panel - "Workout Plan" button navigates to /workout-plan
- WorkoutPlanPage - shows difficulty badge, duration, and exercise table (sets/ reps/ notes)

Summary

Pattern	Category	Feature	New Files	Observer Events
State	Behavioural	Membership Management	8	membershipTransitioned
Builder	Creational	Workout Plan Builder	5	planCreated